OPERATING SYSTEMS

Complete MCA Exam-Oriented Notes

Unit-wise topics • Simple 2–3 line explanations • Algorithms • Comparisons • Important questions

Purpose: Quick revision and exam preparation for MCA-level Operating Systems.

How to Use These Notes

1. Read the 2–3 line explanation of every topic first. 2. Memorize definitions and key differences. 3. Practice the scheduling, Banker’s Algorithm, page replacement, and disk scheduling numericals. 4. Use the question bank at the end for exam revision.

High-priority areas: CPU Scheduling, Process Synchronization, Deadlocks/Banker’s Algorithm, Paging & Page Replacement, File Allocation, and Disk Scheduling.

UNIT 1 — INTRODUCTION TO OPERATING SYSTEMS

Operating System

An Operating System (OS) is system software that acts as an interface between users/applications and computer hardware. It manages resources such as CPU, memory, files, and I/O devices.

Objectives and Functions

The main goals are convenience, efficient resource utilization, reliability, and security. Major functions include process management, memory management, file management, device management, and protection.

Types of Operating Systems

Common types include batch, multiprogramming, multitasking, time-sharing, distributed, network, and real-time operating systems. Each type is designed for a particular workload or environment.

OS Services

The OS provides services such as program execution, I/O operations, file manipulation, communication, error detection, resource allocation, accounting, and protection.

System Calls

System calls provide a controlled interface through which programs request services from the OS kernel. Examples include process creation, file operations, memory allocation, and device access.

Operating System Structure

An OS can be organized using structures such as simple, layered, microkernel, modular, and hybrid designs. The structure affects reliability, performance, security, and maintainability.

Kernel

The kernel is the core part of an operating system that runs with privileged access. It manages CPU scheduling, memory, devices, system calls, and other critical resources.

User Mode and Kernel Mode

User mode provides restricted execution for applications, while kernel mode allows privileged operations. Switching to kernel mode occurs when a program makes a system call or when certain hardware events occur.

Booting

Booting is the process of starting a computer and loading the operating system into memory. Firmware/bootloader performs initialization and then transfers control to the OS kernel.

Multiprogramming

Multiprogramming keeps multiple programs in memory so the CPU can switch to another program when one is waiting for I/O. This improves CPU utilization.

Multitasking

Multitasking allows multiple tasks to appear to run at the same time by rapidly switching the CPU among them. Modern operating systems use scheduling to provide responsive execution.

Time Sharing

Time-sharing gives each process a small CPU time slice, allowing many interactive users or programs to share the processor. It aims for quick response and fairness.

Distributed Systems

A distributed system uses multiple networked computers that cooperate to perform tasks. Resources and processing can be shared across machines.

Real-Time Operating System

An RTOS is designed to respond to events within specified timing constraints. Hard real-time systems must meet deadlines, while soft real-time systems prioritize timely responses.

UNIT 2 — PROCESS MANAGEMENT AND CPU SCHEDULING

Process and Program

A program is a passive set of instructions stored on storage, whereas a process is a program currently in execution. A process has resources such as registers, memory, and an execution state.

Process States

Typical states are New, Ready, Running, Waiting/Blocked, and Terminated. A process moves between these states as it is admitted, scheduled, waits for I/O, or completes.

Process Control Block (PCB)

PCB is a data structure maintained by the OS for each process. It stores process state, program counter, CPU registers, scheduling information, memory information, and accounting details.

Process Scheduling

Process scheduling selects a process from the ready queue to execute on the CPU. The scheduler tries to balance CPU utilization, throughput, turnaround time, waiting time, and response time.

Context Switching

A context switch saves the state of the currently running process and loads the saved state of another process. It enables multitasking but introduces overhead.

Threads

A thread is the smallest unit of CPU execution within a process. Threads of the same process share code, data, and files but normally have their own registers and stack.

Multithreading

Multithreading allows multiple threads within a process to execute concurrently. It can improve responsiveness and resource sharing, especially in interactive and server applications.

Inter-Process Communication (IPC)

IPC allows processes to exchange data and coordinate activities. Common methods are shared memory and message passing.

CPU Scheduling Criteria

Important criteria include CPU utilization, throughput, turnaround time, waiting time, response time, and fairness. Lower waiting/turnaround/response time is generally desirable, while utilization and throughput should be high.

FCFS Scheduling

First-Come, First-Served executes processes in their arrival order. It is simple and non-preemptive, but a long process can make short processes wait for a long time.

SJF Scheduling

Shortest Job First selects the process with the smallest CPU burst. It can minimize average waiting time when burst lengths are known, but long processes may suffer starvation.

SRTF Scheduling

Shortest Remaining Time First is the preemptive form of SJF. If a newly arriving process has a shorter remaining burst than the running process, the CPU may be preempted.

Priority Scheduling

The process with the highest priority is selected for execution. It can be preemptive or non-preemptive; starvation can be reduced using aging.

Round Robin

Round Robin gives each ready process a fixed time quantum in cyclic order. It is preemptive and commonly used for time-sharing systems.

Multilevel Queue

The ready queue is divided into separate queues based on process type or priority. Each queue can have its own scheduling algorithm.

Multilevel Feedback Queue

Processes can move between queues according to their behavior and priority. It is flexible and can favor interactive jobs while preventing indefinite CPU monopolization.

UNIT 3 — PROCESS SYNCHRONIZATION AND DEADLOCKS

Process Synchronization

Synchronization coordinates concurrent processes or threads so shared data remains consistent. It prevents incorrect results caused by uncontrolled concurrent access.

Race Condition

A race condition occurs when multiple processes access shared data concurrently and the final result depends on the order of execution. Proper synchronization is needed to avoid it.

Critical Section

A critical section is the part of a program where shared resources are accessed. Mutual exclusion, progress, and bounded waiting are the classical requirements for a correct critical-section solution.

Mutex

A mutex is a locking mechanism that allows only one thread/process to enter a protected critical section at a time. It is commonly used to protect shared data.

Semaphore

A semaphore is a synchronization variable accessed through atomic operations such as wait and signal. A binary semaphore can provide mutual exclusion, while a counting semaphore can manage multiple identical resources.

Monitors

A monitor is a high-level synchronization construct that encapsulates shared data and procedures. It provides controlled access and can use condition variables for waiting and signaling.

Classical Synchronization Problems

Important examples include Producer–Consumer, Readers–Writers, and Dining Philosophers. They illustrate issues such as mutual exclusion, synchronization, starvation, and deadlock.

Deadlock

Deadlock is a situation where a set of processes are permanently waiting for resources held by one another. None of the involved processes can proceed without external intervention.

Necessary Conditions for Deadlock

The four Coffman conditions are mutual exclusion, hold and wait, no preemption, and circular wait. Deadlock requires all four conditions to exist simultaneously.

Deadlock Prevention

Prevention ensures that at least one necessary deadlock condition is eliminated. For example, resource ordering can prevent circular wait.

Deadlock Avoidance

Avoidance dynamically checks whether granting a resource request can keep the system in a safe state. The Banker’s Algorithm is a well-known avoidance method.

Banker's Algorithm

The Banker’s Algorithm checks available resources and each process's maximum possible demand. A request is granted only if the resulting state remains safe.

Deadlock Detection

Detection allows deadlocks to occur and then identifies them using resource-allocation information. Separate methods can be used for single or multiple instances of resources.

Deadlock Recovery

Recovery can involve terminating one or more processes or preempting resources. The OS chooses victims while considering cost and system progress.

UNIT 4 — MEMORY MANAGEMENT AND VIRTUAL MEMORY

Memory Management

Memory management controls how main memory is allocated and released for processes. It also tracks which memory areas are free or occupied.

Logical and Physical Address

A logical address is generated by the CPU and belongs to a process's address space. A physical address identifies the actual location in main memory after address translation.

Contiguous Memory Allocation

In contiguous allocation, each process is placed in one continuous memory region. Fixed or variable partitions can be used, but fragmentation can occur.

Fixed Partitioning

Main memory is divided into fixed-size partitions and each partition can hold a process. It is simple but may cause internal fragmentation.

Variable Partitioning

Partitions are created according to process requirements. It reduces internal fragmentation but can produce external fragmentation.

Paging

Paging divides logical memory into fixed-size pages and physical memory into fixed-size frames. Pages can be placed in any available frame, reducing external fragmentation.

Page Table

A page table maps a process's page numbers to physical frame numbers. The Memory Management Unit (MMU) uses this mapping during address translation.

Segmentation

Segmentation divides a program into logical variable-sized units such as code, data, and stack. Each segment has a base address and a limit.

Virtual Memory

Virtual memory allows a process to use an address space larger than available physical memory. Parts of a process can remain on secondary storage and be loaded when needed.

Demand Paging

In demand paging, a page is loaded into memory only when it is referenced. If the page is absent, a page fault occurs.

Page Fault

A page fault occurs when a process references a page that is not currently in physical memory. The OS loads the page from secondary storage and updates the page table.

FIFO Page Replacement

FIFO replaces the page that has been in memory for the longest time. It is simple but can suffer from Belady’s anomaly.

Optimal Page Replacement

Optimal replacement removes the page whose next use is farthest in the future. It gives the minimum possible page faults for a reference string but requires future knowledge, so it is mainly a theoretical benchmark.

LRU Page Replacement

Least Recently Used replaces the page that has not been used for the longest recent period. It approximates the locality of reference but may require extra tracking.

Thrashing

Thrashing occurs when a system spends excessive time handling page faults and swapping pages instead of executing processes. It is often associated with insufficient frames and high multiprogramming.

UNIT 5 — FILE SYSTEM AND STORAGE MANAGEMENT

File System

A file system organizes and manages files on secondary storage. It provides naming, storage, access, protection, and directory services.

File Attributes

Common attributes include name, type, size, location, owner, permissions, timestamps, and protection information. These help the OS manage and protect files.

File Operations

Typical operations include create, open, read, write, seek, close, delete, and truncate. These operations are usually exposed through system calls.

File Types

Operating systems may distinguish regular files, directories, executable files, device files, and other special types. The exact types depend on the OS.

File Access Methods

Sequential access processes data in order, while direct/random access allows data to be accessed at specific positions. Indexed access uses an index to locate records.

Directory Structure

Directories organize files into structures such as single-level, two-level, tree, and graph-based directories. They help users locate and manage files.

Contiguous File Allocation

A file occupies consecutive disk blocks. It provides fast sequential and direct access but can cause external fragmentation and may make file growth difficult.

Linked File Allocation

Each file block contains a pointer to the next block. It avoids external fragmentation but direct access is slower because blocks may be scattered.

Indexed File Allocation

An index block contains pointers to the blocks of a file. It supports direct access but requires additional space for index information.

Free Space Management

The OS tracks unused disk blocks using methods such as bitmaps, linked lists, grouping, and counting. This helps allocate space efficiently.

Disk Structure

A magnetic disk contains platters, tracks, sectors, and cylinders. Disk access time includes seek time, rotational latency, and transfer time.

Disk Scheduling

Disk scheduling decides the order in which pending disk requests are served. The goal is usually to reduce head movement and improve response time.

FCFS Disk Scheduling

Requests are served in their arrival order. It is simple and fair but can cause large total head movement.

SSTF Disk Scheduling

Shortest Seek Time First serves the request closest to the current head position. It can reduce seek time but may starve distant requests.

SCAN

SCAN moves the disk head in one direction serving requests and then reverses direction. It is often called the elevator algorithm.

C-SCAN

Circular SCAN services requests in one direction only and, after reaching the end, returns to the beginning without servicing requests on the return trip.

LOOK

LOOK is similar to SCAN but reverses direction at the last pending request rather than going all the way to the physical end of the disk.

C-LOOK

C-LOOK is the circular version of LOOK. The head services requests in one direction and jumps from the last request to the first pending request.

UNIT 6 — PROTECTION, SECURITY AND ADVANCED OS CONCEPTS

Protection

Protection controls how processes and users access system resources. Its purpose is to ensure that authorized operations are allowed while unauthorized access is restricted.

Access Control

Access control defines who can access a resource and what operations they may perform. Permissions can commonly include read, write, execute, or similar rights.

Authentication

Authentication verifies the identity of a user or system. Passwords, biometrics, tokens, and multi-factor authentication are common methods.

Authorization

Authorization determines what an authenticated user or process is allowed to do. It is different from authentication, which verifies identity.

Operating System Security

OS security protects system resources from unauthorized access, malicious software, data theft, and misuse. It involves authentication, permissions, isolation, updates, and monitoring.

Malware and Threats

Malware includes harmful software such as viruses, worms, trojans, ransomware, and spyware. Threats can exploit software vulnerabilities or user mistakes.

Encryption

Encryption converts readable information into protected ciphertext using an algorithm and key. Decryption converts it back into readable form for authorized users.

Virtualization

Virtualization creates virtual versions of computing resources such as machines, storage, or networks. A hypervisor can allow multiple virtual machines to share physical hardware.

Virtual Machines

A virtual machine is an isolated software environment that behaves like a computer. It can run an operating system independently while using resources provided by the host system.

Cloud Operating Systems

Cloud environments manage computing resources distributed across data centers and provide scalable services. Resource virtualization and orchestration are important components.

Distributed Operating Systems

A distributed OS coordinates multiple networked computers and attempts to provide users with a unified environment. It can support resource sharing and distributed processing.

Linux/Unix Basics

Linux/Unix systems commonly use a hierarchical file system, processes, permissions, shells, and command-line utilities. Important concepts include users, groups, files, directories, and processes.

Windows OS Basics

Windows provides process, memory, file, device, security, and graphical user-interface services. It uses components such as the Windows kernel, services, drivers, and NTFS file system.

IMPORTANT COMPARISONS

Process vs Thread

Process: independent execution unit with its own address space; heavier creation/context-switch cost. Thread: execution unit within a process; shares process resources and is generally lighter.

Program vs Process

Program: passive instructions stored on storage. Process: active instance of a program currently executing.

Paging vs Segmentation

Paging uses fixed-size pages/frames and is mainly physical-memory oriented. Segmentation uses variable-sized logical segments such as code, data, and stack.

FCFS vs Round Robin

FCFS executes in arrival order and is normally non-preemptive. Round Robin uses a time quantum and preempts processes for interactive fairness.

Deadlock Prevention vs Avoidance

Prevention structurally eliminates at least one necessary deadlock condition. Avoidance examines each allocation and grants it only when the system remains safe.

Mutex vs Semaphore

A mutex is mainly used for exclusive ownership of a critical section. A semaphore is a synchronization counter that can represent one or multiple available resources.

Internal vs External Fragmentation

Internal fragmentation is unused space inside an allocated block. External fragmentation is free space scattered between allocated blocks.

Logical vs Physical Address

Logical addresses are generated from a process address space. Physical addresses identify actual RAM locations after translation.

IMPORTANT ALGORITHMS — QUICK REVISION

FCFS

Arrange by arrival time → execute each process completely → calculate completion, turnaround and waiting time.

SJF

Choose the ready process with the smallest CPU burst. Non-preemptive SJF waits for the selected job to finish.

SRTF

At each scheduling decision, select the ready process with the smallest remaining CPU burst; preempt if required.

Round Robin

Place ready processes in a queue → give each process a fixed time quantum → move unfinished processes to the back of the queue.

Priority

Select the ready process with the highest priority. State clearly whether smaller or larger number means higher priority.

Banker’s Algorithm

Compute Need = Max – Allocation → find a process whose Need ≤ Available → simulate completion → release its Allocation → repeat to determine a safe sequence.

FIFO Page Replacement

Replace the oldest page in memory. Maintain pages in arrival order.

LRU Page Replacement

Replace the page that has not been used for the longest time in the past.

Optimal Page Replacement

Replace the page whose next use is farthest in the future. Use it as a benchmark for minimum possible faults.

SSTF Disk Scheduling

From the current head position, select the pending request with the smallest seek distance.

SCAN

Move in one direction servicing requests, then reverse direction and continue servicing.

C-SCAN

Service requests in one direction; after the end, jump to the beginning and continue in the same direction.

LOOK

Like SCAN, but reverse at the final pending request rather than at the physical disk end.

C-LOOK

Like C-SCAN, but jump from the last pending request directly to the first pending request.

SHORT-ANSWER QUESTIONS

1. What is an operating system? State any four functions.
2. What is a system call? Give examples.
3. What is a PCB?
4. Differentiate process and thread.
5. What is context switching?
6. Define race condition and critical section.
7. State the four necessary conditions for deadlock.
8. What is a semaphore?
9. What is paging?
10. What is a page fault?
11. What is thrashing?
12. Define file allocation.
13. What is disk scheduling?
14. Differentiate authentication and authorization.
15. What is virtualization?

LONG-ANSWER / NUMERICAL QUESTIONS

1. Explain OS services and system calls with suitable examples.
2. Explain process states and draw the process-state transition diagram.
3. Solve a CPU scheduling problem using FCFS, SJF, SRTF, Priority, and Round Robin and calculate waiting/turnaround times.
4. Explain threads, multithreading, and IPC.
5. Explain critical-section requirements and semaphore-based synchronization.
6. Explain deadlock and its four necessary conditions. Discuss prevention, avoidance, detection, and recovery.
7. Solve a Banker’s Algorithm problem and determine whether the system is safe.
8. Explain paging, page tables, address translation, and virtual memory.
9. Solve a page-replacement reference string using FIFO, Optimal, and LRU and calculate page faults.
10. Explain file allocation methods with diagrams and compare their advantages/disadvantages.
11. Solve a disk-scheduling problem using FCFS, SSTF, SCAN, C-SCAN, LOOK, and C-LOOK and calculate total head movement.
12. Explain protection, authentication, authorization, and common OS security threats.
13. Explain virtualization, virtual machines, and the role of hypervisors.

Final Exam Tip

For theory answers, use: **Definition** → **Explanation** → **Example/Diagram** → **Advantages/Disadvantages (if applicable)** → **Conclusion.** For numerical algorithms, always show the table/Gantt chart, intermediate steps, and final calculation.